

```

using UnityEngine;
using UnityEngine.AI; // Acceso al sistema de navegaci3n
using UnityEngine.SceneManagement; // Importado para reiniciar la escena
using _Scripts.Systems; // Acceso al coraz3n de datos (ScriptableObject)
using _Scripts.Interfaces; // Acceso al contrato IEstado
using _Scripts.States; // Acceso a las l3gicas de estado (Rastreo, Sigilo...)

namespace _Scripts.Brains
{
    /* * * CLASE: EnemyBrain
    * DESCRIPCI3N: Act3a como el "Cuerpo" y "Contexto" de la IA (Hardware).
    * No decide qu3 hacer por s3 mismo, sino que delega esa tarea a un objeto IEstado.
    * Mantiene vivo el cron3metro de 60s para el enfriamiento.
    * * * * * */
    public class EnemyBrain : MonoBehaviour
    {
        [Header("Referencias de Datos")]
        [Tooltip("El archivo de datos (SO) que compartimos con el jugador y el sistema")]
        [SerializeField] private IntuicionSystem data;
        public IntuicionSystem Data => data; // Propiedad p3blica para los Estados

        [Header("Configuraci3n de Movimiento y Navegaci3n")]
        [Tooltip("Agente de navegaci3n del enemigo")]
        [SerializeField] private NavMeshAgent agent;
        public NavMeshAgent Agent => agent; // Propiedad p3blica para los Estados

        [Tooltip("Velocidad de traslaci3n f3sica base.")]
        [SerializeField] private float velocidadBase = 3.5f;

        [Tooltip("Puntos de patrulla para el movimiento perpetuo.")]
        [SerializeField] private Transform[] waypoints;
        public Transform[] Waypoints => waypoints; // Propiedad p3blica para los Estados

        [Header("Instintos Base")]
        [Tooltip("Transform del jugador para el Olfato")]
        [SerializeField] private Transform transformJugador;
        [Tooltip("Distancia a la que el enemigo mata instant3neamente")]
        [SerializeField] private float distanciaOlfato = 1.5f;

        /* L3GICA DE M3QUINA DE ESTADOS e OPTIMIZACI3N (CACHING) */
        private IEstado _estadoActual;

        // Cacheamos las instancias de los estados para evitar asignaciones de memoria (GC
        Allocations)
        private IEstado _estadoRastreo;

        [Header("Reloj de Recuperaci3n")]
        // Contador de segundos para llegar al minuto de enfriamiento
        private float _cronometroSesentaSeg = 0f;

        // Guardamos el nivel de fase actual (1, 2, 3...) para pas3rsele al SO en el Tick
        private int _nivelMiedoActual = 1;

        /* * *
        * M3TODO: Start
        * Se ejecuta al nacer. Configuramos los estados cacheando referencias y el estado
        inicial.
        */
        private void Start()
        {
            if (!data) return; // Si no hay datos sal de la funci3n.

            // Verificaci3n y asignaci3n autom3tica de componentes obligatorios
            if (!agent) agent = GetComponent<NavMeshAgent>();
            if (!agent) Debug.LogError("EnemyBrain: Falta el componente NavMeshAgent.");
            if (!transformJugador) Debug.LogError("EnemyBrain: Falta asignar el Transform d
            el Jugador para el Olfato.");

```

```

        // Suscripción al SO
        data.OnSubirFase += HandleSubirFase;
        data.OnBajarFase += HandleBajarFase;

        // OPTIMIZACIÓN\223N: Instanciamos los estados una sola vez en el inicio
        _estadoRastreo = new EstadoRastreo(this);

        // ESTADO INICIAL: Iniciamos usando la referencia cacheada
        CambiarEstado(_estadoRastreo);
    }

    /* * *
     * MÃ211TODO: Update
     * Ejecución por frame. Mantenemos el pulso de los relojes y la máquina de estado
s.
    */
    private void Update()
    {
        if (!data) return;

        // 1. CONDICIÓN\223N GLOBAL (OLFATO): Instinto base primario
        if (transformJugador != null && Vector3.Distance(transform.position, transformJ
ugador.position) <= distanciaOlfato)
        {
            EjecutarMuertePorOlfato();
            return;
        }

        // 2. Manejamos el tiempo para el enfriamiento de 60s
        ManejarCronometroRecuperacion();

        // 3. EJECUCIÓN\223N DEL ESTADO: El estado controla la lógica en este frame
        _estadoActual?.Ejecutar();
    }

    private void EjecutarMuertePorOlfato()
    {
        Debug.Log("<color=red>Â;JUGADOR ASESINADO POR OLFATO! Reiniciando escena...</co
lor>");

        // Detenemos el movimiento fÃ3sico inmediatamente
        if (agent != null && agent.isOnNavMesh)
        {
            agent.isStopped = true;
        }

        // Reiniciamos la escena actual
        SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
    }

    /* * *
     * MÃ211TODO: ActualizarVelocidad
     * Función pública para que los ESTADOS puedan cambiar la velocidad del NavMeshAg
ent.
    */
    public void ActualizarVelocidad(float nuevaVelocidad)
    {
        velocidadBase = nuevaVelocidad;
        if (agent != null)
        {
            agent.speed = nuevaVelocidad;
        }
    }

    /* * *
     * MÃ211TODO: MoverHacia
     * Función pública para que los ESTADOS ordenen el destino al NavMeshAgent.

```

```
    */
    public void MoverHacia(Vector3 destino)
    {
        if (agent != null && agent.isOnNavMesh)
        {
            agent.SetDestination(destino);
        }
    }

    /* * *
    * MÃ\211TODO: CambiarEstado
    * TransiciÃ³n limpia de estados reutilizando referencias existentes.
    */
    public void CambiarEstado(IEstado nuevoEstado)
    {
        _estadoActual?.Salir();
        _estadoActual = nuevoEstado;
        _estadoActual.Entrar();
    }

    /* * *
    * MÃ\211TODO: ManejarCronometroRecuperacion
    * Controla el enfriamiento por Ticks.
    */
    private void ManejarCronometroRecuperacion()
    {
        _cronometroSesentaSeg += Time.deltaTime;

        if (_cronometroSesentaSeg >= 60f)
        {
            data.EjecutarTickDeRecuperacion(_nivelMiedoActual);
            _cronometroSesentaSeg = 0f;
        }
    }

    // --- MANEJADORES DE EVENTOS ---

    private void HandleSubirFase()
    {
        _nivelMiedoActual++;
        Debug.Log($"<color=orange>RADIO: El enemigo subiÃ³ al Nivel {_nivelMiedoActual}
    </color>");
        // Nota: AquÃ- se cambiarÃ; al estado correspondiente cacheado en el futuro
    }

    private void HandleBajarFase()
    {
        _nivelMiedoActual = Mathf.Max(1, _nivelMiedoActual - 1);

        // Si bajamos al nivel base, volvemos a usar la referencia cacheada de Rastreo
        if (_nivelMiedoActual == 1) CambiarEstado(_estadoRastreo);

        Debug.Log($"<color=cyan>RADIO: El enemigo bajÃ³ al Nivel {_nivelMiedoActual}</c
    olor>");
    }

    /* * *
    * MÃ\211TODO DE LIMPIEZA: OnDestroy
    * Se ejecuta si matamos al enemigo o cambiamos de escena.
    * Desconecta los cables de radio para no causar errores de memoria.
    */
    private void OnDestroy()
    {
        if (data != null)
        {
            data.OnSubirFase -= HandleSubirFase;
            data.OnBajarFase -= HandleBajarFase;
        }
    }
}
```

```
    }  
}  

```